

[HW-02] Domain Testing Report

1. General

1.1 Information

- Student name: Dinh Hoang Nam
- Student ID: 23127430
- Class: 23KTPM1
- Github Repository: <https://github.com/namdin05/ST-Group>

1.2 Selected Feature

Pool	Feature ID	Feature Name	Platform
A	FR-01	Account Registration	Web
B	FR-07	Shopping Cart	Web
C	FR-13	Dashboard	Web Admin
D	FR-06	Product Detail	Mobile

2. Content

2.1 FR-01: Account Registration

Stage A – Domain variables

Variable Type	Variable	Description	Constraint / Relation	Scope
Input	Họ Tên / Name	User's display name entered in the registration form	Required; must be accepted by both UI and API	UI and API
Input	Email	User's email address	Required; must have a valid email format and be unique in the system	UI and API
Input	Mật khẩu / Password	User-chosen password	Required; must be strong: at least 8 characters, at least 1 uppercase, 1 lowercase, 1 digit, and 1 special character from @, \$, !, %, *, ?, &	UI and API
Input	Xác nhận mật khẩu / Confirm password	Re-entry of password in the UI	Must match the password exactly; UI-only field, not part of the API contract	UI
Input	Form submit action	User presses the registration button	Must validate inputs before sending and then navigate to login on success	UI
Input	Registration request body	JSON payload sent to <code>POST /api/register</code>	Contains <code>name</code> , <code>email</code> , <code>password</code>	API
System state	Existing user records	Previously registered accounts in the database	Email uniqueness must be enforced across all users	API
System state	Session/navigation state	Current route and auth-free registration flow	Successful registration must redirect to the login page	UI
Output	Validation message	Error feedback displayed by the UI or returned by the API	Must explain the violated rule without exposing sensitive internals	UI and API
Output	Registration success response	Backend confirmation after valid registration	Returns success message and created user id	API
Output	Redirect target	Post-submit navigation after successful registration	Must go to <code>/login</code>	UI
Cross-variable constraint	Password confirmation	Password and confirm password relationship	The two values must be identical before submission is accepted	UI
Cross-variable constraint	Email uniqueness	Submitted email versus stored emails	Duplicate email must be rejected	UI and API
Cross-variable constraint	Password policy	Length and composition of password	Must satisfy all strength rules simultaneously	UI and API
Cross-variable constraint	Client/server consistency	UI validation versus API validation	API must still reject invalid payloads if the request bypasses the UI	API

Stage B – Equivalence partitions

Partition ID	Description	Valid or invalid	Representative value	Requirement basis	Risk	Type
UI-V1	Valid name, valid email, strong password, confirm password matches	Valid	Nguyen Van A, user@example.com, Password123!, Password123!	FR-01 registration criteria	High	Specification-derived
UI-V2	Empty name field	Invalid	''	FR-01 requires Họ Tên	Medium	Specification-derived
UI-V3	Invalid email format	Invalid	user.example.com	FR-01 email format rule	High	Specification-derived
UI-V4	Weak password missing one or more required classes	Invalid	Password12	FR-01 strong password rule	High	Specification-derived
UI-V5	Password and confirm password do not match	Invalid	Password123! vs Password123?	FR-01 confirm password rule	High	Specification-derived
UI-V6	Duplicate email already registered	Invalid	existing@domain.com	FR-01 email uniqueness rule	High	Specification-derived
UI-R1	Very long but otherwise well-formed inputs	Exploratory robustness	300-character name, long local-part email, long password	No explicit max length in FR-01	Medium	Exploratory
API-V1	Complete valid registration payload	Valid	{name,email,password} with strong password	API contract for /api/register	High	Specification-derived
API-I1	Missing name	Invalid	{email,password}	Registration requires Họ Tên	High	Specification-derived
API-I2	Missing email	Invalid	{name,password}	Registration requires Email	High	Specification-derived
API-I3	Missing password	Invalid	{name,email}	Registration requires Mật khẩu	High	Specification-derived
API-I4	Invalid email format in JSON body	Invalid	user.example.com	Email format requirement	High	Specification-derived
API-I5	Weak password in JSON body	Invalid	Password12 or password123!	Password policy requirement	High	Specification-derived
API-I6	Duplicate email in JSON body	Invalid	existing@domain.com	Email uniqueness requirement	High	Specification-derived
API-R1	Extra unexpected JSON fields	Exploratory robustness	{name,email,password,role:"admin"}	No explicit support for extra fields	Medium	Exploratory

Stage C – Boundary analysis

Boundary	Just below	At boundary	Just above
Password length	7 characters	8 characters	9 characters
Password uppercase presence	0 uppercase letters	1 uppercase letter	2 uppercase letters
Password lowercase presence	0 lowercase letters	1 lowercase letter	2 lowercase letters
Password digit presence	0 digits	1 digit	2 digits
Password special character presence	0 special characters from the allowed set	1 allowed special character	2 allowed special characters
Confirm password equality	One character different	Exact match	Same value with trailing extra character
Email uniqueness	Existing email in database	First use of a new email	Second use of the same email
Email format	Missing @ or domain part	user@domain.com	Email with additional subdomain/plus-tag variation that is still valid
Name presence	Empty string	One visible character	Two visible characters
Input size robustness	Short valid value	Moderate valid value	Very long value beyond ordinary UI usage

Stage D – Combination strategy

The test design uses one-factor-at-a-time for the UI flow so that each failure can be tied to a single user-facing rule: required name, email format, password policy, confirm-password match, duplicate email handling, and success navigation. The baseline UI case is a fully valid registration submission; each additional test changes only one dominant factor whenever possible.

For the API, the baseline is a valid JSON registration payload. Decision-table thinking is used to combine the most important backend checks without creating a full Cartesian product: valid payload, missing field, invalid email, weak password, duplicate email, and extra-field robustness. UI-only confirm-password behavior is kept separate because the API contract does not include that field. Very long input cases are isolated as exploratory robustness tests so no unstated maximum length is implied.

Stage E – Test cases

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data
TC001	Account Registration	Web	UI	Domain partition	FR-01	Verify a fully valid registration submits successfully	User is on the register page; email is not yet registered	Nguyen Van A, newuser@example.com, Password
•	Password is not follow the rule	FAILED	AI Generated	Reviewed	Baseline success case			
TC002	Account Registration	Web	UI	Equivalence partition	FR-01	Verify the form rejects an empty name	User is on the register page	`, newuser@example.com, Password
TC003	Account Registration	Web	UI	Equivalence partition	FR-01	Verify the form rejects an invalid email format	User is on the register page	Nguyen Van A, newuser.example.com, Password
TC004	Account Registration	Web	UI	Boundary value analysis	FR-01	Verify the minimum strong-password length boundary	User is on the register page	Nguyen Van A, newuser@example.com, Passw1!,
TC005	Account Registration	Web	UI	Boundary value analysis	FR-01	Verify the minimum strong-password length boundary	User is on the register page	Nguyen Van A, newuser@example.com, Password
TC006	Account Registration	Web	UI	Decision table	FR-01	Verify confirm-password mismatch is blocked	User is on the register page	Nguyen Van A, newuser@example.com, Password
TC007	Account Registration	Web	UI	Equivalence partition	FR-01	Verify duplicate email is rejected in the UI flow	An account already exists with existing@domain.com	Nguyen Van A, existing@domain.com, Password
TC008	Account Registration	Web	UI	Exploratory robustness test	FR-01	Observe UI behavior for unusually	User is on the register page	300-character name, long valid email, password, matching confirm password

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data
						long but valid-looking input		
TC009	Account Registration	Web	API	Domain partition	FR-01	Verify API registration accepts a complete valid payload	API client can send POST requests; email is not yet registered	{ "name": "Nguyen Van A", "email": "newuser@example.com", "password": "Password123!" }
TC010	Account Registration	Web	API	Invalid payload	FR-01	Verify missing name is rejected by the backend	API client can send POST requests	{ "email": "newuser@example.com", "password": "Password123!" }
TC011	Account Registration	Web	API	Invalid payload	FR-01	Verify missing email is rejected by the backend	API client can send POST requests	{ "name": "Nguyen Van A", "password": "Password123!" }
TC012	Account Registration	Web	API	Invalid payload	FR-01	Verify missing password is rejected by the backend	API client can send POST requests	{ "name": "Nguyen Van A", "email": "newuser@example.com" }
TC013	Account Registration	Web	API	Equivalence partition	FR-01	Verify invalid email format is rejected by the backend	API client can send POST requests	{ "name": "Nguyen Van A", "email": "newuser.example.com", "password": "Password123!" }
TC014	Account Registration	Web	API	Boundary value analysis	FR-01	Verify password length at the minimum boundary	API client can send POST requests	{ "name": "Nguyen Van A", "email": "newuser@example.com", "password": "1234567890" }
TC015	Account Registration	Web	API	Boundary value analysis	FR-01	Verify a password meeting the minimum boundary is accepted when other fields are valid	API client can send POST requests	{ "name": "Nguyen Van A", "email": "newuser@example.com", "password": "1234567890" }
TC016	Account Registration	Web	API	Equivalence partition	FR-01	Verify duplicate email is rejected by	An account already exists with existing@domain.com	{ "name": "Nguyen Van A", "email": "existing@domain.com", "password": "Password123!" }

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data
TC017	Account Registration	Web	API	Exploratory robustness test	FR-01	the backend		
						Observe backend behavior for extra unexpected fields	API client can send POST requests	{ "name": "Nguyen Van A", "email": "newuser@example.com", "password": "Password123!", "role": "admin" }
TC018	Account Registration	Web	UI	Security	FR-01	Kiểm tra lỗ hổng XSS tại trường Họ Tên		Name: <script>alert(1)</script>

2.2 FR-07: Shopping Cart

Stage A – Domain variables

Variable Type	Variable	Description	Constraint / Relation	Scope
Input	Cart item list	The set of products currently in the cart	Can be empty or populated; same product must map to one logical line item	UI and API
Input	Product identity	Product selected from Home or Product Detail	Must reference an existing product; same product should not create duplicate lines	UI and API
Input	Quantity adjustment	User action to increase or decrease quantity	Quantity changes by one step using plus/minus controls	UI
Input	Delete action	User triggers item removal	Removal must be preceded by a confirmation dialog	UI
Input	Continue shopping action	User clicks the link/button to return to Home	Must navigate back to the home page	UI
Input	Authorization header	JWT header for cart API calls	Required for /api/cart endpoints	API
Input	Cart item payload	JSON body sent to POST /api/cart	Expected fields: id, name, price, quantity	API
System state	Cart ownership	Cart is associated with a user token	Cart data must be isolated per user	API
System state	Empty cart state	Cart contains zero items	Must render empty-state feedback	UI and API
Output	Rendered table	Cart rows and column headings	Must show Sản phẩm, Đơn giá, Số lượng, Thành tiền, Thao tác	UI
Output	Total amount	Sum of all line totals	Must display the correct cart total and the label Tổng cộng	UI
Cross-variable constraint	Same-product merge	Same product added repeatedly	Must increase quantity instead of creating a new row	UI and API
Cross-variable constraint	Deletion confirmation	Delete intent versus confirmed deletion	Only confirmed deletion mutates the cart	UI
Cross-variable constraint	Cart isolation	Two users accessing cart data	One user must not see another user's cart	API

Stage B – Equivalence partitions

Partition ID	Description	Valid or invalid	Representative value	Requirement basis	Risk	Type
UI-V1	Empty cart state renders empty message and shopping link	Valid	0 items	FR-07 empty cart requirement	Medium	Specification-derived
UI-V2	Cart shows one or more distinct products with required columns	Valid	1 product line	FR-07 table columns	High	Specification-derived

Partition ID	Description	Valid or invalid	Representative value	Requirement basis	Risk	Type
UI-V3	Same product added twice should merge into one line with increased quantity	Valid	2 adds of the same product	FR-07 same-product rule	High	Specification-derived
UI-V4	Quantity can be increased or decreased through +/- controls	Valid	Quantity 1 and 2	FR-07 quantity adjustment	High	Specification-derived
UI-I1	Delete action without confirmation does not remove item	Invalid	Cancel in confirmation dialog	FR-07 delete confirmation	High	Specification-derived
UI-V5	Continue shopping returns the user to Home	Valid	Click continue shopping	FR-07 continue shopping requirement	Medium	Specification-derived
API-V1	Authorized GET /api/cart returns current cart for the signed-in user	Valid	Bearer token with empty or populated cart	Cart API auth requirement	High	Specification-derived
API-V2	Authorized POST /api/cart accepts a complete item payload	Valid	{"id":1,"name":"iPhone 15 Pro Max","price":30000000,"quantity":1}	API contract for cart add	High	Specification-derived
API-I1	Missing or malformed Authorization header	Invalid	No token or wrong scheme	Cart API auth requirement	High	Specification-derived
API-I2	Partial or malformed cart payload	Invalid	Missing quantity, non-numeric quantity, or quantity 0	API schema and quantity behavior	High	Specification-derived
API-V3	Cart data is isolated per user token	Valid	User A token vs User B token	Per-user cart scope	High	Specification-derived
API-R1	Very large quantity value for robustness observation	Exploratory	999999	No explicit upper bound in spec	Medium	Exploratory

Stage C – Boundary analysis

Boundary	Just below	At boundary	Just above
Empty versus populated cart	0 items	1 item	2 items
Line-item quantity	0	1	2
Same-product merge count	0 repeated adds	1 repeated add	2 repeated adds
Authorization header presence	Missing header	Valid Bearer token	Malformed Bearer token
Cart line total	0 đ	Price of 1 item × quantity 1	Price of 1 item × quantity 2

Stage D – Combination strategy

The test design uses one-factor-at-a-time around a stable baseline so the cart’s core behaviors stay traceable without creating a full Cartesian product. For UI coverage, the baseline is a cart with one seeded product and the factors are empty versus populated state, same-product repetition, quantity adjustment, delete confirmation, and navigation back to Home.

For API coverage, the baseline is a valid JWT and a complete item payload. The key factor combinations are token presence, header format, payload completeness, quantity validity, and user isolation. Decision-table thinking is used for the most important interactions: valid token plus valid payload, valid token plus duplicate add, missing or malformed token, and invalid payload values. The exploratory large-quantity case is kept separate from the boundary set so no unstated maximum is implied.

Stage E – Test cases

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps	Expected Results
TC001	Shopping Cart	Web	UI	Domain partition	FR-07	Verify the empty-cart presentation	Cart is empty	Empty cart	Open /cart	Empty cart with no items
TC002	Shopping Cart	Web	UI	Domain partition	FR-07	Verify cart table layout	Cart contains one seeded	iPhone 15 Pro Max x1	Open /cart	Table contains one row with product details

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps	Expected Results
						and total label	product			Đảm bảo tính toàn vẹn của dữ liệu
TC003	Shopping Cart	Web	UI	Equivalence partition	FR-07	Verify same-product merging in the UI	Cart is empty; product is available	Add product id 1 twice	Add the same product twice, then open cart	On the cart page, the product is listed only once.
TC004	Shopping Cart	Web	UI	Boundary value analysis	FR-07	Verify quantity increment behavior	Cart has one item with quantity 1	Quantity 1, then click +	Open cart and click + once	Quantity increases from 1 to 2.
TC005	Shopping Cart	Web	UI	Boundary value analysis	FR-07	Verify quantity decrement behavior around the minimum logical quantity	Cart has one item at quantity 1 and one item at quantity 2 for comparison	Quantity 1 and 2, then click -	Open cart and click - on the quantity control	Quantity decreases from 2 to 1, but does not go below 1.
TC006	Shopping Cart	Web	UI	Decision table	FR-07	Verify delete confirmation flow	Cart has one item	One cart item	Click Xóa, cancel once, then confirm on the second attempt	Cart item is successfully removed after confirmation.
TC007	Shopping Cart	Web	UI	State transition	FR-07	Verify continue-shopping navigation	Any cart state	Any cart contents	Click Tiếp tục mua sắm	User is redirected to the product page without losing cart contents.
TC008	Shopping Cart	Web	API	Domain partition	FR-07	Verify cart retrieval for an authenticated user with an empty cart	Valid JWT for a user with no cart items	Authorization: Bearer valid token	Send GET /api/cart	200 OK response with an empty cart array.
TC009	Shopping Cart	Web	API	Equivalence partition	FR-07	Verify authorized add-to-cart with a complete payload	Valid JWT; product exists	{"id":1,"name":"iPhone 15 Pro Max","price":30000000,"quantity":1}	Send POST /api/cart, then GET /api/cart	Item is added to the cart successfully.
TC010	Shopping Cart	Web	API	Equivalence partition	FR-07	Verify duplicate add of the same product is merged logically	Valid JWT; cart starts empty	Same product payload posted twice	Send POST /api/cart twice with the same product, then GET /api/cart	Only one instance of the product is in the cart.

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps	Expected Results
TC011	Shopping Cart	Web	API	Invalid header	FR-07	Verify missing or malformed Authorization header is rejected	No valid token supplied	Missing header or Authorization: Token abc	Send GET /api/cart or POST /api/cart	Reject or status code 401
TC012	Shopping Cart	Web	API	Invalid payload	FR-07	Verify malformed cart payload is rejected	Valid JWT	Missing quantity, non-numeric quantity, or quantity 0	Send POST /api/cart with invalid body	Reject with non-200 status
TC013	Shopping Cart	Web	API	State isolation	FR-07	Verify cart isolation between two users	Two distinct valid JWTs	User A adds item; user B checks cart	User A posts an item; User B sends GET /api/cart	User B cannot see User A's items
TC014	Shopping Cart	Web	API	Exploratory robustness test	FR-07	Observe server behavior for an unusually large quantity value	Valid JWT; product exists	{"id":1,"name":"iPhone 15 Pro Max","price":30000000,"quantity":999999}	Send POST /api/cart and inspect the stored value	System behavior should be OK and reviewed as mentioned in spec

2.3 FR-13: Dashboard

Stage A – Domain variables

A.1 UI-level domain variables

- Admin authentication state: logged in / logged out.
- Active page state: dashboard selected / other admin tabs selected.
- Orders dataset loaded into the dashboard.
- Per-order attributes:
 - `status` with the business values `pending`, `confirmed`, `shipping`, `delivered`, `canceled`.
 - `total_amount` as the monetary amount stored for each order.
- Dashboard outputs:
 - Total revenue card.
 - Total orders card.
 - Visible dashboard title and labels.

A.2 API-level domain variables

- Request authorization header: present / missing / invalid / non-admin.
- `GET /api/admin/orders` response payload.
- Order records returned by the API:
 - `id`
 - `status`
 - `total_amount`
 - `user_id`
 - `user_name` if joined by the backend
- Seed/data setup state:
 - No orders
 - Mixed order statuses
 - Delivered-only orders
 - Large-volume orders

A.3 Outputs

- `total_revenue = sum(total_amount of orders where status = delivered)`
- `total_orders = count(all orders returned to admin)`
- Revenue display formatted with thousands separators and `₹`

A.4 Cross-variable constraints

- Only orders with `status = delivered` contribute to revenue.
- All orders contribute to total order count, regardless of status.
- Dashboard access depends on admin authentication and role authorization.
- Dashboard must not show non-delivered orders in the revenue calculation.
- Revenue display must remain readable for zero, small, and large totals.
- UI and API results must stay consistent for the same underlying order dataset.

Stage B – Equivalence partitions

Partition ID	Description	Valid or invalid	Representative value	Requirement basis	Risk	Type
FR13-EP-01	Delivered orders exist and have positive amounts	Valid	2 delivered orders: 100000 and 250000	FR-13 revenue rule	Core revenue computation	Specification-derived
FR13-EP-02	Mixed statuses exist; only delivered orders should count	Valid	1 delivered, 1 pending, 1 canceled	FR-13 revenue rule	Incorrect inclusion of non-delivered orders	Specification-derived
FR13-EP-03	No orders exist	Valid	Empty order list	FR-13 total order count	Empty-state handling	Specification-derived
FR13-EP-04	Orders exist but none are delivered	Valid	3 orders: pending/confirmed/shipping	FR-13 revenue rule	Revenue should be 0, not error	Specification-derived
FR13-EP-05	Admin token missing	Invalid	No <code>Authorization</code> header	FR-12 dependency for admin features	Access control failure	Specification-derived
FR13-EP-06	Token present but role is not admin	Invalid	User token	FR-12 dependency for admin features	Unauthorized dashboard access	Specification-derived
FR13-EP-07	Order record has malformed or missing status	Invalid	<code>status = null</code>	Backend/data integrity assumption	Aggregation failure or undefined behavior	Exploratory robustness
FR13-EP-08	Order record has malformed or missing total amount	Invalid	<code>total_amount = null</code>	Backend/data integrity assumption	Revenue calculation failure	Exploratory robustness
FR13-EP-09	Very large delivered totals or many orders	Valid	Large integer amount / large list	System-level scalability	Overflow, truncation, display issues	Exploratory robustness
FR13-EP-10	Zero-value delivered order	Valid	<code>total_amount = 0</code>	Monetary aggregation edge case	Shows whether zero is handled cleanly	Exploratory robustness

Stage C – Boundary analysis

Boundary	Just below	At boundary	Just above
Delivered order count contributing to revenue	0 delivered orders? no contribution from delivered set is the lower edge for revenue inclusion	1 delivered order	2 delivered orders
Total order count shown on dashboard	-1 orders, only as an exploratory invalid dataset	0 orders	1 order
Delivered order total amount	-1 ₹, exploratory invalid data	0 ₹	1 ₹
Aggregate revenue value	-1 ₹, exploratory invalid data	0 ₹	1 ₹
Large display value for formatting	999,999,999 ₹, exploratory robustness	1,000,000,000 ₹, exploratory robustness	1,000,000,001 ₹, exploratory robustness
Order status filter boundary	Any non-delivered status such as <code>shipping</code>	<code>delivered</code>	Another non-delivered status such as <code>canceled</code>

Notes:

- FR-13 does not define explicit numeric limits, so the numeric boundaries above are partly exploratory robustness tests.
- The only explicit business boundary in FR-13 is the inclusion boundary for `status = delivered`.

Stage D – Combination strategy

I use a decision-table approach combined with one-factor-at-a-time variation.

For the dashboard, the outcome depends mainly on two independent factors:

- the composition of the order list by status
- the sum of `total_amount` within the delivered subset

Instead of testing every possible permutation of statuses and amounts, I cover representative combinations:

- empty dataset
- all non-delivered orders
- mixed statuses with one delivered order
- mixed statuses with multiple delivered orders
- large-value delivered orders for formatting robustness

For UI tests, I keep the order dataset fixed per case and vary one factor at a time, such as:

- count only
- delivered-only revenue
- formatting of the displayed total
- empty state

For API tests, I verify the source data returned by `GET /api/admin/orders` and confirm that the dashboard rule can be computed correctly from that payload. This avoids a full Cartesian product while still covering the business-critical paths.

Stage E – Test cases

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps
TC001	Dashboard	Web Admin	UI	Domain Testing	FR-13, FR-12	Verify dashboard shows correct revenue and total order count for a mixed dataset	Admin is logged in; dashboard is open; database contains delivered and non-delivered orders	2 delivered orders: 100000, 250000; 1 pending order: 70000; 1 canceled order: 80000	Open Dashboard tab
TC002	Dashboard	Web Admin	UI	Boundary Value Analysis	FR-13	Verify zero-order boundary is handled cleanly	Admin is logged in; database has no orders	Empty order list	Open Dashboard tab
TC003	Dashboard	Web Admin	UI	Domain Testing	FR-13	Verify revenue includes only delivered orders when all other statuses are present	Admin is logged in; dashboard is open	3 orders: pending 100000, confirmed 200000, shipping 300000	Open Dashboard tab
TC004	Dashboard	Web Admin	UI	Exploratory robustness	FR-13	Verify large delivered totals display with thousands separators and <code>₹</code>	Admin is logged in; dashboard is open	1 delivered order: 1000000000	Open Dashboard tab

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps
TC005	Dashboard	Web Admin	API	Domain Testing	FR-13, FR-12	Verify admin order source data can be retrieved for dashboard aggregation	Valid admin token is available	GET /api/admin/orders with mixed order statuses in DB	Send authenticated request to /api/admin/orders
TC006	Dashboard	Web Admin	API	Access control / negative	FR-13, FR-12	Verify admin orders endpoint is protected	No token or non-admin token is used	GET /api/admin/orders without valid admin authorization	Send request without Authorization header or with non-admin token
TC007	Dashboard	Web Admin	API	Boundary Value Analysis	FR-13	Verify dashboard source data is correct when all orders are non-delivered	Valid admin token is available	GET /api/admin/orders with only pending/confirmed/shipping orders	Send authenticated request inspect returned payload
TC008	Dashboard	Web Admin	API	Exploratory robustness	FR-13	Verify response-driven aggregation remains stable with a very large delivered amount	Valid admin token is available	One delivered order with a very large total_amount	Send authenticated request to /api/admin/orders

2.4 FR-06: Product Detail

Stage A - Domain variables

A1. Input variables

Layer	Variable	Domain / meaning	Constraint source
UI	Product selection / product id	Product selected from the mobile product list before opening the detail screen	FR-06, FR-05
UI	Quantity text input	User-entered quantity on the product detail screen	FR-06
UI	Add-to-cart action	Tap on the add button after the detail screen is loaded	FR-06
API	id path parameter in GET /api/products/:id	Product identifier used to fetch detail data	API spec 3.2
API	Request body in POST /api/cart	Payload used to add an item to the server cart	API spec 4.2
API	Authorization header	Token required for cart API access	API spec 4.1 and backend route guard

A2. System state variables

Layer	State variable	Domain / meaning	Why it matters
UI	Current view	Home, product detail, or cart	Controls whether the user can inspect detail data and add to cart
UI	Product fetch result	Loaded product, empty object, or network/error state	Determines whether the detail screen renders the product or a fallback state
UI	Cart contents	Empty cart, cart with one item, cart with same item already present	Needed to verify add-to-cart behavior and quantity aggregation
API	Product existence	Existing row vs missing row in <code>products</code> table	Controls the response of <code>GET /api/products/:id</code>
API	Authentication state	Valid token vs missing/invalid token	Required for <code>POST /api/cart</code>
API	Cart storage state	Existing server cart array vs new server cart array	Affects duplicate add behavior on the backend

A3. Output variables

Layer	Output	Expected shape
UI	Product detail screen	Image, name, price, description, category information, quantity field, add button
UI	Feedback after add	Toast, alert, or badge/cart update that confirms the add action
UI	Cart change	New line item or incremented quantity for the same product
API	Product detail response	JSON object for an existing product, or graceful not-found style response for missing product
API	Cart response	Success message or validation/auth error with correct HTTP status

A4. Cross-variable constraints

Constraint ID	Constraint
C-01	Quantity must be a positive integer and must be at least 1
C-02	The add-to-cart action must use the selected product and the current quantity value together
C-03	Opening the detail screen must not mutate cart contents
C-04	Adding the same product again must increment its quantity instead of creating an unrelated product entry
C-05	<code>GET /api/products/:id</code> must only read product data and must not change state
C-06	<code>POST /api/cart</code> must require a valid token and a complete payload

Stage B - Equivalence partitions

Partition ID	Description	Valid or invalid	Representative value	Requirement basis	Risk	Type (Specification-derived or Exploratory)
UI-P-01	Existing product selected from the list and detail data loads normally	Valid	Product id <code>1</code>	FR-06 detail view	Core happy path may fail if navigation or fetch is broken	Specification-derived
UI-P-02	Quantity is a positive integer	Valid	<code>1</code>	FR-06 quantity minimum	Add-to-cart must accept the only explicit valid lower bound	Specification-derived
UI-P-03	Same product already exists in cart	Valid	Product id <code>1</code> already in cart	FR-06 plus cart consistency	Quantity aggregation may be implemented incorrectly	Specification-derived
UI-P-04	Product does not exist or fetch returns an empty object	Invalid	Product id <code>999999</code>	System-level handling of missing product data	App may crash or show a blank screen	Exploratory
UI-P-05	Quantity is <code>0</code>	Invalid	<code>0</code>	FR-06 minimum boundary	Quantity below the minimum must	Specification-derived

Partition ID	Description	Valid or invalid	Representative value	Requirement basis	Risk	Type (Specification-derived or Exploratory)
					not be accepted	
UI-P-06	Quantity is negative	Invalid	-1	FR-06 positive integer rule	Negative quantity may be normalized or mishandled	Specification-derived
UI-P-07	Quantity is a non-integer value	Invalid	1.5	Positive integer rule	Decimal input may bypass weak parsing	Exploratory
UI-P-08	Quantity is non-numeric text or blank	Invalid	abc / empty string	Positive integer rule	Text input can reach the parser through the mobile keyboard	Exploratory
API-P-01	GET /api/products/:id with an existing id returns product data	Valid	GET /api/products/1	API spec 3.2	Failure blocks the detail screen	Specification-derived
API-P-02	GET /api/products/:id with a missing or malformed id	Invalid	GET /api/products/0	Product id must refer to an existing row	Missing product handling may be inconsistent	Specification-derived
API-P-03	POST /api/cart with valid token and complete item body	Valid	{id:1,name:"...",price:30000000,quantity:1}	API spec 4.2	Core server-side cart path may fail	Specification-derived
API-P-04	POST /api/cart without Authorization header	Invalid	No token	API spec 4.1 and backend authentication	Security boundary may be bypassed	Specification-derived
API-P-05	POST /api/cart with partial or tampered JSON body	Invalid	Missing quantity or price	Bypass UI validation directly to backend	Backend may accept malformed data	Exploratory
API-P-06	POST /api/cart with invalid quantity value	Invalid	quantity: 0 or -2	Positive integer rule	Server-side validation may be missing	Specification-derived

Stage C - Boundary analysis

Boundary	Just below	At boundary	Just above
Quantity minimum	0	1	2
Quantity sign / integer rule	-1 or 1.5	1	2
Product id access boundary, using the lowest seeded row as the minimum existing id	0	1	2
Cart payload completeness boundary	Missing required field such as quantity	Full required body: id, name, price, quantity	Full body plus an extra unrelated field

Notes:

- The quantity boundary is specification-derived because FR-06 explicitly states that quantity must be a positive integer with a minimum of 1.
- The product-id boundary is an implicit system boundary derived from the database's autoincrement primary key and the seeded dataset.
- The cart payload boundary is a system-level schema boundary for bypassing the mobile UI and testing the backend directly.

Stage D - Combination strategy

The UI test set uses one-factor-at-a-time around a single valid baseline: a real product opens correctly, quantity is 1, and the add button is tapped once. From that baseline, each negative case changes only one driver at a time: product existence, quantity class, or cart state. This avoids a full Cartesian product of product data, quantity values, and screen states.

The API test set uses a small decision table instead of combining every product, quantity, and token combination. The key decisions are: product exists or not, token present or missing, and payload complete or tampered. That gives coverage of the server-side boundary conditions without multiplying equivalent cases that would

exercise the same code path.

Stage E - Test cases

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps	Expected Result	Acceptance Criteria
TC001	FR-06 Mobile Product Detail	Mobile	UI	Domain Testing	FR-06	Verify that a valid product opens the detail screen with all required fields visible	Mobile app is running; product list is loaded; product id 1 exists	Product id 1	1. Open the product list. 2. Tap a valid product. 3. Observe the detail screen.	The app shows the product image, name, price, description, and category information without crashing.	
TC002	FR-06 Mobile Product Detail	Mobile	UI	Domain Testing	FR-06	Verify that the quantity field starts at the minimum valid value	Detail screen for an existing product is open	Default quantity state	1. Open a product detail screen. 2. Observe the quantity field before editing.	The quantity field is initialized to 1 .	
TC003	FR-06 Mobile Product Detail	Mobile	UI	Boundary Value Analysis	FR-06	Verify the exact minimum valid quantity	Detail screen for an existing product is open	Quantity 1	1. Enter 1 in the quantity field. 2. Tap Add to cart.	The product is added successfully and the confirmation feedback is shown.	
TC004	FR-06 Mobile Product Detail	Mobile	UI	Boundary Value Analysis	FR-06	Verify that quantity 0 is not accepted	Detail screen for an existing product is open	Quantity 0	1. Enter 0 in the quantity field. 2. Tap Add to cart.	The system does not accept a quantity below 1; it either blocks the add action or normalizes the value to the minimum valid quantity.	The system accepts a quantity of 0
TC004	FR-06 Mobile Product Detail	Mobile	UI	Boundary Value Analysis	FR-06	Verify that negative quantity is not accepted	Detail screen for an existing product is open	Quantity -1	1. Enter -1 in the quantity field. 2. Tap Add to cart.	The system does not accept the value as a valid quantity below 1.	The system accepts -1
TC005	FR-06 Mobile Product Detail	Mobile	UI	Exploratory Robustness Test	FR-06	Verify that non-integer input does not bypass the positive-integer rule	Detail screen for an existing product is open	Quantity 1.5 or abc	1. Enter a decimal or text value. 2. Tap Add to cart.	The system does not add the item with a non-integer quantity; it should keep the quantity valid or show validation feedback.	The system accepts text
TC006	FR-06 Mobile	Mobile	UI	Domain Testing	FR-06	Verify that adding the same	Same product is already	Product id 1 , quantity 1 then 1 again	1. Open the same product	The cart keeps one line item for	

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps	Expected Result	Actual Result
	Product Detail					product twice updates the existing cart entry	present in the cart		detail twice. 2. Add it to cart each time. 3. Check the cart contents.	that product and increases its quantity.	
TC007	FR-06 Mobile Product Detail	Mobile	API	Domain Testing	FR-06 / API 3.2	Verify that a valid product id returns a complete detail payload	Backend API is running; product id 1 exists	GET /api/products/1	1. Send the request directly to the backend. 2. Inspect the response body.	The API returns 200 OK with the product record needed for the detail screen.	
TC008	FR-06 Mobile Product Detail	Mobile	API	Boundary Value Analysis	FR-06 / API 3.2	Verify the missing-product boundary at and around the lowest id	Backend API is running	GET /api/products/0, GET /api/products/1, GET /api/products/2	1. Send requests for a below-boundary id, the lowest seeded id, and the next id. 2. Compare the responses.	The API handles the below-boundary or missing id gracefully and returns the expected record for existing ids.	
TC009	FR-06 Mobile Product Detail	Mobile	API	Domain Testing	FR-06 / API 4.2	Verify that a valid cart-add request is accepted	Valid JWT token is available	POST /api/cart with a full item body and quantity: 1	1. Send the request directly to the backend with a valid token. 2. Inspect the response.	The API accepts the request and returns a success response.	
TC010	FR-06 Mobile Product Detail	Mobile	API	Decision Table / Robustness	FR-06 / API 4.2	Verify that cart-add requests with missing auth or invalid quantity are rejected	Backend API is running	Missing token, partial JSON body, or quantity: 0 / -2	1. Send the request without Authorization or with a tampered body. 2. Inspect the response status and body.	The API rejects the request with a validation or authorization error and does not mutate cart state.	API does not reject quantity < 1
TC011	FR-06 Mobile Product Detail	Mobile	UI	Exploratory	FR-06	Xác minh tính ổn định của UI khi gặp sự kiện gián đoạn vòng đời ứng dụng		Sản phẩm ID 1	1. Mở xem chi tiết sản phẩm ID 1.2. Giả lập một cuộc gọi đến (hoặc ẩn app xuống nền). 3. Quay lại ứng dụng sau 10 giây.	Ứng dụng không bị crash, giao diện chi tiết sản phẩm giữ nguyên trạng thái và nút "Thêm vào giỏ hàng" vẫn hoạt động bình thường.	
TC012	FR-06 Mobile Product Detail	Mobile	UI	BVA	FR-06	Kiểm tra biên hiển thị (Text Overflow) với tên sản phẩm cực dài		Sản phẩm có tên dài 200 ký tự	1. Chọn mở sản phẩm có tên siêu dài từ danh sách. 2. Quan sát màn hình chi tiết.	Tên sản phẩm không làm che khuất hoặc đẩy nút "Thêm vào giỏ hàng" ra	

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps	Expected Result	Ac Re
										khởi màn hình.	